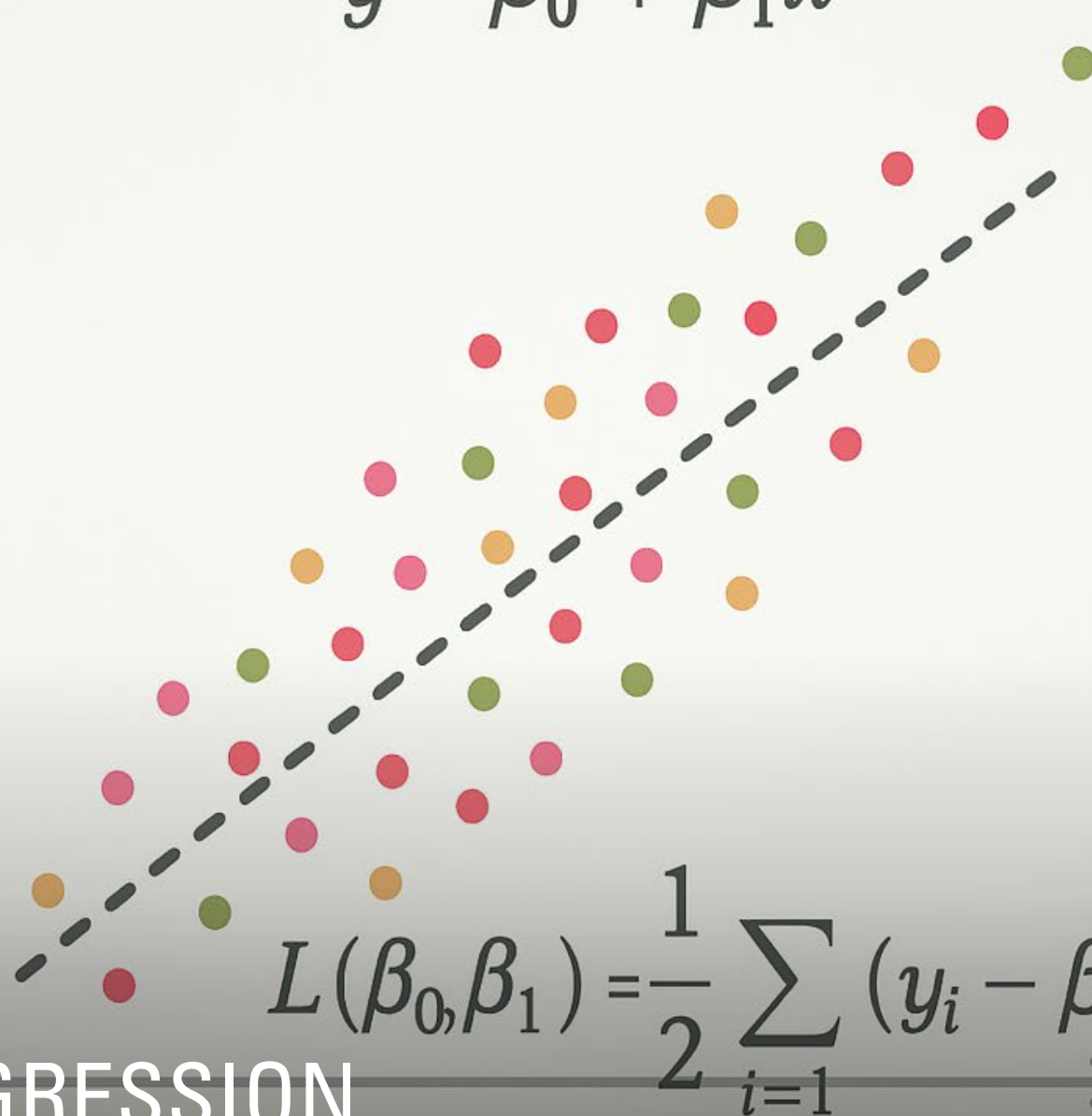


$$y = \beta_0 + \beta_1 x$$



INTRO TO REGRESSION

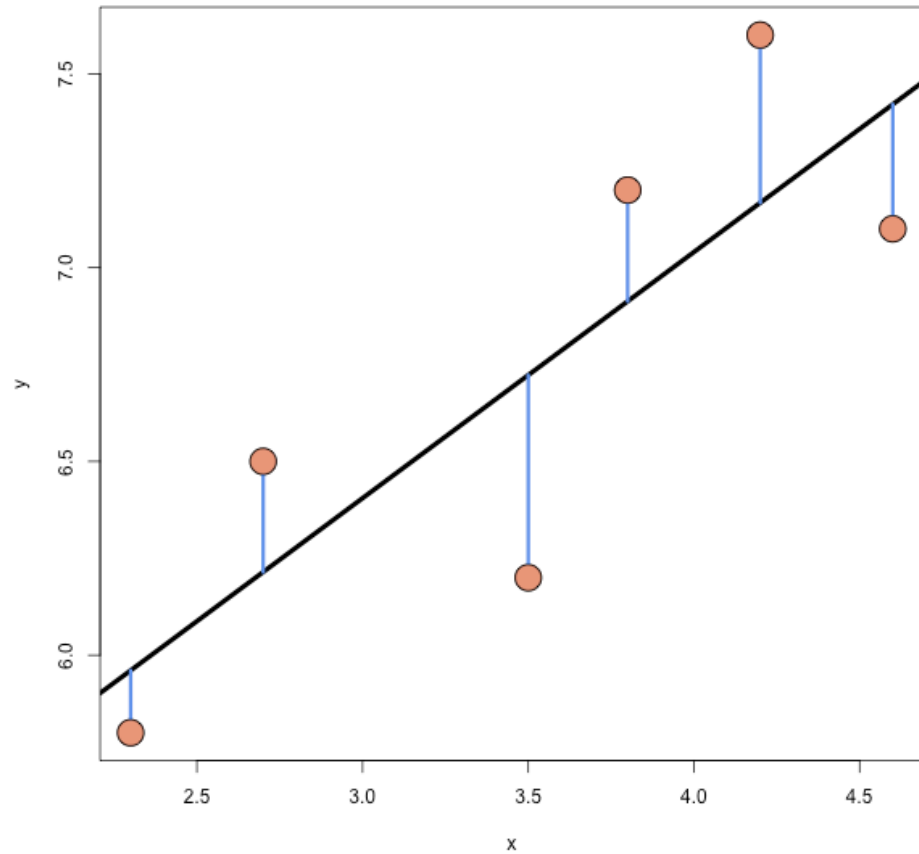
Linear Regression

LINEAR REGRESSION

- ▶ Linear regression is a **supervised** learning approach for predicting a **quantitative** response
- ▶ Linear regression has been around for a long time
 - Still very widely used
 - Best method for truly linear relationships
 - Many "fancier" methods are based on linear regression

Training a Linear Regression Model

LEAST SQUARES



Least squares line is the line that minimizes

$$L(\boldsymbol{\beta}) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

OBJECTIVE FUNCTIONS AND OPTIMIZATION

- **Define an appropriate objective function**
 - Cost/loss functions are minimized (machine learning)
 - Minimum vs. minimizer: the minimizer is the value x that produces the smallest output $f(x)$.
 - **“Train” the model by choosing an optimization method**
 - A closed-form solution (rare) — differential calculus
 - A numerical method (usually), when no analytic solution exists
 - Gradient descent or variants (Adam, Newton’s method)
-

TYPICAL FORMS OF LINEAR REGRESSION MODELS

Scalar Form

- $y_i = \beta_0 + \sum_{j=1}^p \beta_j x_{ij} + \varepsilon_i$ for $i = 1, 2, \dots, n$

Matrix Notation

- $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$

Assumptions

- Linearity
 - Homoscedasticity
 - Independence of Errors
 - Normal Residuals
 - No Multicollinearity
-

ORDINARY LEAST SQUARES OBJECTIVE

- We want $\hat{y} = X\beta$ to be a *good* approximation..
- How do we measure goodness?
- We want $y - \hat{y}$ to be *small*.
- Why use $(y - \hat{y})^2$ and not $|y - \hat{y}|$?
- The objective function:

$$J(\beta) = \frac{1}{2n} (y - X\beta)^T (y - X\beta)$$

METHOD 1: NORMAL EQUATIONS

- Matrix notation cost function:

$$J(\beta) = \frac{1}{2n} (y - X\beta)^T (y - X\beta)$$

- Optimal $\hat{\beta}$:

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

METHOD 2: NUMERICAL APPROXIMATIONS

- To be continued...!

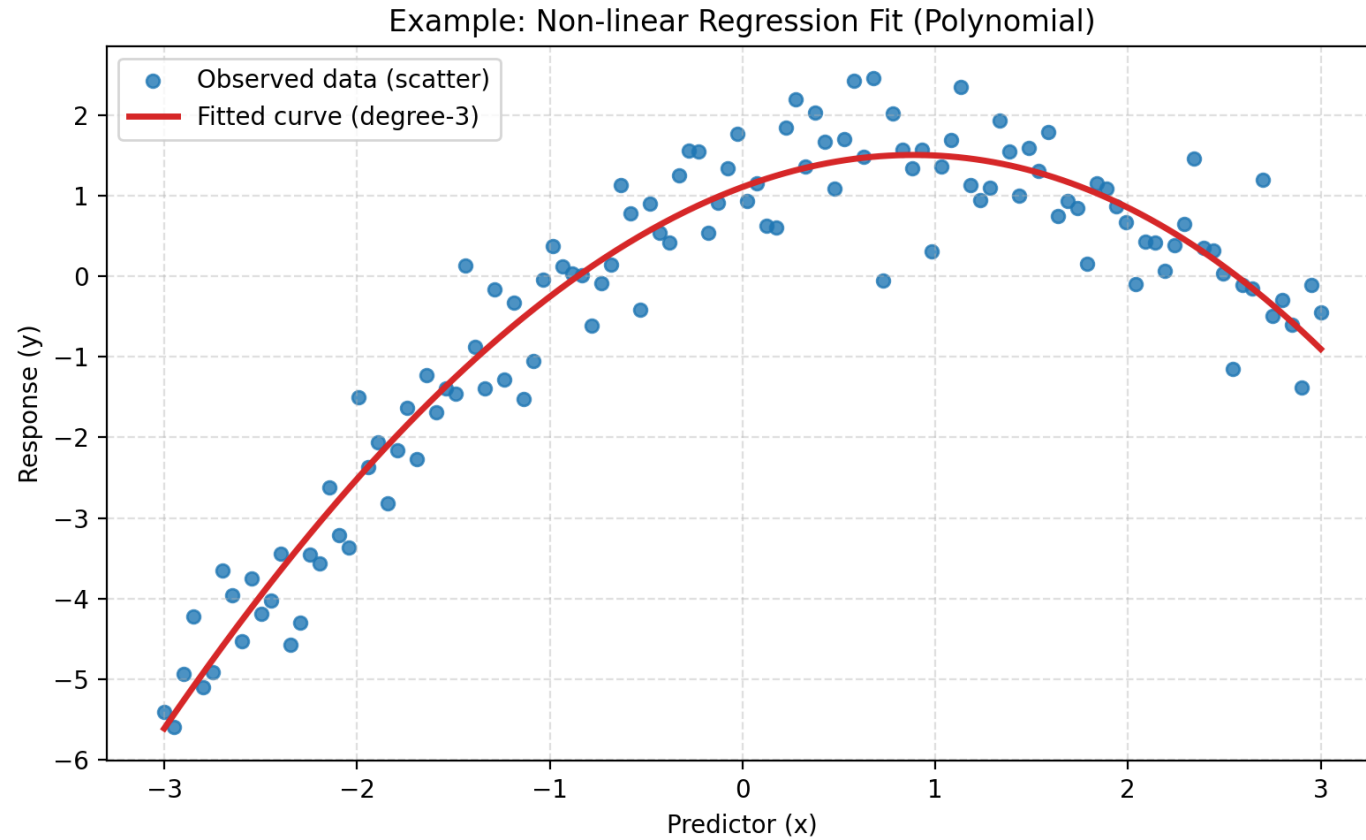
“NONLINEAR” LINEAR REGRESSION

POLYNOMIAL REGRESSION

- Sometimes the relationship between X and y is non-linear
- We can use linear regression to model non-linear relationships (between X and y)
 - Add quadratic, cubic, etc. terms to the model
 - This is sometimes called polynomial regression
 - (Now we are moving into the realm of multiple regression, where we have more than one predictor)

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \epsilon_i$$

POLYNOMIAL REGRESSION



$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \epsilon_i$$

TERMINOLOGY

- Linear regression is "linear" with respect to the parameters (the model is a linear combination of the parameters)

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \epsilon_i$$

- True "nonlinear" regression is where the regression model is a nonlinear combination of the parameters, for example:

$$y_i = \frac{\beta_1 x_i}{\beta_0 + x_i} + \epsilon_i$$

MORE TERMINOLOGY

▸ Just FYI:

- **Multiple** regression is a regression model with more than one predictor (or feature)
- **Multivariate** regression is a regression model with more than one response (or target)

INTERACTION & POLYNOMIAL FEATURES

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

# Add squared and cubed features for every predictor plus interactions
poly = PolynomialFeatures(degree=3)
X = poly.fit_transform(X)

# Only include up to 3rd-order interactions
inter = PolynomialFeatures(degree=3, interaction_only=True)
X = inter.fit_transform(X)
```

REGRESSION METRICS

EVALUATING A REGRESSION MODEL

- In regression (we are predicting a numeric variable),
what does a “good” prediction look like?
- Ideally, $\hat{y}$, (the prediction) is close to y (the observed value)

COMMON REGRESSION METRICS

- Lower is better:

- Mean absolute error (MAE): $MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$

- Mean squared error (MSE): $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

- Root mean squared error (RMSE): $RMSE = \sqrt{MSE}$

- Higher is better:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

REGRESSION METRIC COMPARISONS

MAE Mean Absolute Error

- + Easy to interpret
- + Tolerant of outliers
- Underweights large errors

R² Coefficient of Determination

- + Variance explained
- + Comparable across models
- Misleading for non-linear

MSE Mean Squared Error

- + Penalizes large errors
- + Optimization-friendly
- Sensitive to outliers

RMSE Root Mean Squared Error

- + Same units as target
 - + Highlights large errors
 - Sensitive to outliers
-

Regression metrics

User guide. See the [Regression metrics](#) section for further details.

d2_absolute_error_score	D^2 regression score function, fraction of absolute error explained.
d2_pinball_score	D^2 regression score function, fraction of pinball loss explained.
d2_tweedie_score	D^2 regression score function, fraction of Tweedie deviance explained.
explained_variance_score	Explained variance regression score function.
max_error	The max_error metric calculates the maximum residual error.
mean_absolute_error	Mean absolute error regression loss.
mean_absolute_percentage_error	Mean absolute percentage error (MAPE) regression loss.
mean_gamma_deviance	Mean Gamma deviance regression loss.
mean_pinball_loss	Pinball loss for quantile regression.
mean_poisson_deviance	Mean Poisson deviance regression loss.
mean_squared_error	Mean squared error regression loss.
mean_squared_log_error	Mean squared logarithmic error regression loss.
mean_tweedie_deviance	Mean Tweedie deviance regression loss.
median_absolute_error	Median absolute error regression loss.
r2_score	R^2 (coefficient of determination) regression score function.
root_mean_squared_error	Root mean squared error regression loss.
root_mean_squared_log_error	Root mean squared logarithmic error regression loss.

REGRESSION METRICS IN SCIKIT-LEARN

In v1.4 or greater
(previous versions
have a “squared”
argument in the
MSE function)



MODEL SELECTION METRICS

- Models selection tools, such as “GridSearchCV” and “cross_val_score” use a scoring parameter

For the most common use cases, you can designate a scorer object with the `scoring` parameter; the table below shows all possible values. All scorer objects follow the convention that **higher return values are better than lower return values**. Thus metrics which measure the distance between the model and the data, like `metrics.mean_squared_error`, are available as `neg_mean_squared_error` which return the negated value of the metric.

<code>'neg_mean_absolute_error'</code>	<code><u>metrics.mean_absolute_error</u></code>
<code>'neg_mean_squared_error'</code>	<code><u>metrics.mean_squared_error</u></code>
<code>'neg_root_mean_squared_error'</code>	<code><u>metrics.root_mean_squared_error</u></code>

`'neg_root_mean_squared_error'` requires v1.4+; earlier versions use the “squared” argument
